

Model Checker Aided Design of a Controller for a Wafer Scanner

Hendriks, M.; Nieuwelaar, N.J.M. van den; Vaandrager, F.W.

2006, Article / Letter to editor (International Journal on Software Tools for Technology Transfer, 8, 6, (2006), pp. 633-647)

Doi link to publisher: <https://doi.org/10.1007/s10009-006-0025-7>

Version of the following full text: Author's version preprint

Downloaded from: <https://hdl.handle.net/2066/35654>

Download date: 2026-04-01

Note:

To cite this publication please use the final published version (if applicable).

Model Checker Aided Design of a Controller for a Wafer Scanner[★]

Martijn Hendriks¹, Barend van den Nieuwelaar^{2★★}, and Frits Vaandrager¹

¹ Nijmegen Institute for Computing and Information Sciences,
Radboud University Nijmegen, The Netherlands
{martijnh,fvaan}@cs.ru.nl,

² Department of Mechanical Engineering
Eindhoven University of Technology, The Netherlands
N.J.M.v.d.Nieuwelaar@tue.nl

Abstract. For a case-study of a wafer scanner from the semiconductor industry it is shown how model checking techniques can be used to compute (i) a simple yet optimal deadlock avoidance policy, and (ii) an infinite schedule that optimizes throughput. Deadlock avoidance is studied based on a simple finite state model using SMV, and for throughput analysis a more detailed timed automaton model has been constructed and analyzed using the UPPAAL tool. The SMV and UPPAAL models are formally related through the notion of a stuttering bisimulation. The results were obtained within two weeks, which confirms once more that model checking techniques may help to improve the design process of realistic, industrial systems. Methodologically, the case study is interesting since two models (and in fact also two model checkers) were used to obtain results that could not have been obtained using only a single model (tool).

1 Introduction

Scheduling and resource allocation problems occur in many different domains, for instance (1) scheduling of production lines in factories to optimize costs and delays, (2) scheduling of computer programs in (real-time) operating systems to meet deadline constraints, (3) scheduling of micro instructions inside a processor with a bounded number of registers and processing units, (4) scheduling of trains (or airplanes) over limited quantities of railway tracks and crossroads, and (5) mission planning for autonomous robots on spacecrafts. Typically, in each of these domain problems are solved using different approaches and mathematical tools. The EU IST project Ametist (see <http://ametist.cs.utwente.nl/>) envisages a unifying framework for time-dependent behavior and dynamic resource allocation that crosses the boundaries of application domains.

[★] Supported by the European Community Project IST-2001-35304 (Ametist).

^{★★} Part-time software architect at ASML, Veldhoven, The Netherlands.

In the Ametist approach, components of a system are modeled as *dynamical systems* with a state space and a well-defined dynamics. All that can happen in a system is expressed in terms of *behaviors* that can be generated by the dynamical systems; these constitute the semantics of the problem. Verification, optimization, synthesis and other design activities explore and modify system structure so that the resulting behaviors are correct, optimal, etc. Preferably, the limitations of currently known computational solutions should not influence modeling too much: only after the semantics of a problem is properly understood, abstractions and specialization due to computational considerations can intervene. In such situations, the soundness of abstractions should ideally also be proved, either via deductive verification or model checking.

The mission of Ametist is to extend this approach, which underlies the successful domain of *formal verification*, to resource allocation, scheduling and other time-related problems. The mathematical carrier for the Ametist methodology is the *timed automaton* model [2,3], a modeling framework for discrete event dynamical systems that can handle quantitative timing delays between events. Some tools for *model checking* timed automata already exist, e.g., KRONOS [22] and UPPAAL [13]. Model checking is a method for formally verifying dynamical systems. Specifications about the system are expressed as temporal logic formulas, and efficient symbolic algorithms are used to traverse the model and to check (fully automatically) if the specification holds or not. We aim at further improving model checking tools for timed automata, investigating the applicability of these tools, and establishing links to tools developed in specific domains whenever appropriate.

In this paper, as an illustration of the Ametist methodology, we use model checking techniques to solve the deadlock avoidance and throughput optimization problems for a realistic case of a wafer scanner from the semiconductor industry.

A major concern in the design of controllers for many resource allocation systems (RASs) is *deadlock*, a permanently blocking condition. There are three general ways of handling deadlock: (i) deadlock prevention, (ii) deadlock detection and resolution, and (iii) deadlock avoidance. Deadlock prevention restricts the system in such a way that deadlock is a priori impossible. As a consequence, performance may be unnecessarily low. Deadlock detection and resolution, on the other hand, is not restrictive at all and detects and resolves a deadlock at run-time. This, however, may be very expensive. Deadlock avoidance achieves a middle ground; it dynamically chooses the control actions to avoid the occurrence of deadlock. In this paper, we show how a least restrictive deadlock avoidance policy (DAP) for the wafer scanner can be easily computed using SMV, a model checker for finite automata. This DAP can be represented by a very short predicate over the states of the wafer scanner, which can be used by the controller for the wafer scanner.

In addition, we use the timed automaton tool UPPAAL to define a refined model that adds timing constraints to address the issue of throughput optimization. We relate the UPPAAL model to the SMV model via the concept of *stuttering*

bisimulation introduced by Browne, Clarke and Grumberg [5]. Since stuttering bisimulation preserves validity of CTL formulas (without nexttime operator), all properties (and in particular the DAP) that we established for the untimed model using SMV, carry over to the UPPAAL model. It is not possible to compute the least restrictive DAP directly for the UPPAAL model since (a) UPPAAL does not support full CTL, and (b) the state space of the UPPAAL model is so big that it cannot be fully explored. Using heuristics, however, we are able to use the UPPAAL model checker to find an infinite schedule that optimizes throughput.

Contribution. We obtained our results within two weeks, and we believe that our method can be applied by engineers with a background in computer science after training of only a few days. This confirms that model checking may help to improve the design process of realistic, industrial systems. Our DAP computation approach is referred to in a patent application of ASML, which shows its significance for industry. Methodologically, the case study is interesting since two models (and in fact also two model checkers) were used in combination to obtain results that could not have been obtained using only a single model (tool). Our approach illustrates once more that building models that are just abstract enough for addressing a specific question, often provides a way to deal with the state space explosion problem. The SMV and UPPAAL models are formally related through the notion of a stuttering bisimulation. We are not aware of other work that addresses both deadlock avoidance and throughput optimization in (what essentially is) a single framework.

Related work. Other papers in which model checking tools are used to solve scheduling problems include a case study in which a control schedule for a smart card personalization system is synthesized using the SMV model checker [10], and a case study in which the UPPAAL model checker is used to find feasible schedules for a steel plant [9]. The present work is a follow-up on [4], which considers the same example as the present paper and uses suboptimal deadlock avoidance heuristics to generate schedules that are not guaranteed to be optimal. The present work, however, gives a least restrictive (and thus optimal) DAP and a schedule that optimizes stationary throughput in the absence of errors.

Much research has been devoted to deadlock avoidance in RASs, see for instance [18, 19]. Discouraged by the NP-completeness of optimal deadlock avoidance for many RAS classes, see for instance [14], this kind of work generally focuses either on computation of suboptimal but polynomial DAPs or on optimal policies for very specific sub classes. Much of this work uses the Petri net formalism [17] for the modeling and analysis of RASs.

In [11] a deadlock free controller is constructed by an iterative process. The parallel composition of the controller and the plant is checked against deadlock by SMV. If a deadlock state is found, then the controller is adjusted to exclude the counterexample and the verification is run again. Otherwise, the controller is deadlock free. Finally, the work presented in [21] deals with verification of several DAPs using SMV.

Outline. First, Section 2 informally presents the case study. Section 3 then presents the SMV model and shows two ways of obtaining an optimal DAP us-

ing SMV. In Section 4, a UPPAAL model of the wafer scanner is proposed and infinite schedules which optimize throughput are computed. Finally, Section 5 draws some conclusions and gives directions for future work. A full version of this article, which includes all the proofs, is available as [12]. The complete SMV and UPPAAL models used in our case study are available at the URL <http://www.cs.ru.nl/ita/publications/papers/martijnh/>.

2 The EUV Machine

Lithographic machines, called *wafer scanners*, are used within the semiconductor industry to project chip designs on slices of silicon which are called wafers. A key performance characteristic of wafer scanners is throughput, i.e., the number of wafers that can be processed per time unit. For a typical recipe¹ it is desirable that the exposure operation (which uses the lens which is the most expensive part of the machine) is critical in optimal schedules. In order to maximize throughput, a controller should have a strategy that optimizes throughput in the absence of errors. Furthermore, we require that the controller is deadlock-free, since deadlock resolution is expensive.

Figure 1 schematically depicts a possible design of an *Extreme Ultra Violet machine* (EUV machine), which is a particular type of wafer scanner that is currently being developed by ASML. The inside of an EUV machine is kept vacuum as EUV light is absorbed by air. The wafer flow is presented in Figure 1.

First, the external track robot (which is not shown) puts a wafer in one of the four locks. This lock is depressurized, and then the wafer is picked up by one of the two internal robots. Each internal robot has two arms that can each hold a wafer and that are opposite to each other. The internal robot turns and puts the wafer on the closest chuck, which is in the so-called “measure position”. The wafer is measured and a chuck swap is performed. The chuck with the measured wafer now is in the “expose position” and the wafer is exposed. After another chuck swap, the exposed wafer is picked up by one of the internal robots which turns and puts it in a depressurized lock. After the lock has been pressurized, the track robot removes the exposed wafer from the machine. Each wafer thus has a fixed recipe for its route: lock - internal robot - chuck - internal robot - lock. There is a choice which locks, internal robots

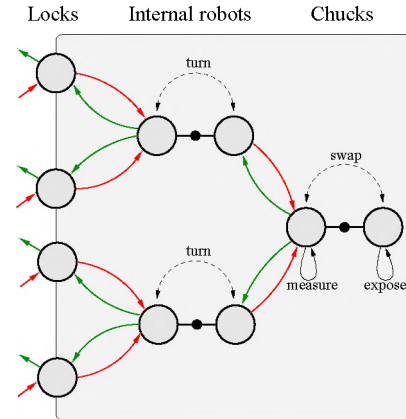


Fig. 1: Wafer paths within the EUV machine.

¹ The timing parameters of the production depend on the chips to be produced.

and chucks are used by a wafer. An obvious question that arises is why we not let the unexposed wafers flow through the upper two locks and let the exposed wafers exit through the lower two locks. In that case there are no crossing material paths which means that there is no deadlock possible by construction. The answer is twofold. First, if locks are unidirectional then filling the machine from the initial, empty, state takes unnecessarily long. Second, if locks are unidirectional then the depressurization operation might become critical instead of the exposure, since depressurization takes more than twice as long as exposure in a typical wafer recipe. As noted above, this is undesirable. In Section 4, we will prove that indeed the exposure subsystem is critical in the design of Figure 1, and that restricting the wafer flow to prevent deadlock a priori lowers both the throughput and the utilization of the exposure subsystem.

A typical example of a deadlock situation in the EUV machine would be a state in which all four robot arms hold unprocessed wafers, and both chucks hold processed wafers. A controller for the EUV machine should ensure that no such deadlock situation can ever be reached. The problem of finding such a control strategy is commonly referred to as the deadlock avoidance problem. The EUV machine is a disjunctive RAS according to the taxonomy of [15]. Instead of the traditional Petri net or graph based approaches to solving the deadlock avoidance problem, we will show in the next section how it can be tackled using the SMV model checker.

3 A Least Restrictive Deadlock Avoidance Policy

In this section, after a (very) brief introduction into SMV, we present our SMV model of the EUV machine, discuss how one can formalize the notion of deadlock as a temporal logic formula, and present the deadlock avoidance policy that we synthesized using SMV. The reader is referred to [7] and [16] for an extensive introduction into model checking and SMV.

3.1 SMV

In the approach supported by the SMV model checker, a system is modeled as a finite *transition system*, i.e. as a tuple $(S, s_{\text{init}}, \rightarrow)$ where S is a finite set of states, s_{init} is the initial state, and $\rightarrow \subseteq S \times S$ is the transition relation. We write $s \rightarrow s'$ instead of $(s, s') \in \rightarrow$. A state is defined as a valuation of a number of *state variables*. The value of state variable v in state s is denoted by $s(v)$. Furthermore, $s[v := c]$ denotes the state that is obtained by updating the value of v in state s to c . A *path* of a transition system is a sequence $s_0 s_1 s_2 \dots$ such that for all i , $s_i \rightarrow s_{i+1}$. A state is *reachable* if it occurs on some path that starts in s_{init} .

In SMV, specifications are described in *Computation Tree Logic (CTL)*, a branching time temporal logic. Below some examples of CTL formulas are given, which should be sufficient to understand the present paper. The basic building blocks of CTL are *atomic formula*, which denote functions from the set of states

to $\{true, false\}$. For instance, if v is a state variable, then $v = 2$ is an atomic formula, which denotes the function from states to $\{true, false\}$ that maps a state s to $true$ iff $s(v) = 2$. In this case, we say state s *satisfies* formula $v = 2$, notation $s \models (v = 2)$. Every atomic formula is a *state formula*. State formulas can be combined with Boolean connectives and *path operators*. We show three path operators that are relevant for this paper. First, if ϕ is a state formula, then $\mathbf{AG}(\phi)$ also is a state formula. A state s satisfies $\mathbf{AG}(\phi)$, denoted by $s \models \mathbf{AG}(\phi)$, if for all paths $s_0s_1s_2\dots$ with $s = s_0$, and for all $i \geq 0$, $s_i \models \phi$. Second, if ϕ is a state formula, then $\mathbf{EF}(\phi)$ is also a state formula. We define $s \models \mathbf{EF}(\phi)$ if there exists a path $s_0s_1s_2\dots$ such that $s = s_0$ and $s_i \models \phi$, for some $i \geq 0$. Finally, if ϕ is a state formula, then $\mathbf{EG}(\phi)$ also is a state formula. We define $s \models \mathbf{EG}(\phi)$ if there exists a path $s_0s_1s_2\dots$ with $s = s_0$ such that for all $i \geq 0$, $s_i \models \phi$.

3.2 An SMV Model of the EUV Machine

The EUV machine can be modeled conveniently and concisely in SMV. In fact, the full code is displayed in Figure 2.

```

module main ()
{
  -- state variables
  l : array 0..3 of {e,r,g};
  rb: array 0..1 of array 0..1 of {e,r,g};
  c : array 0..1 of {e,r,g};

  -- initialization
  for (i=0; i<4; i=i+1)
    init(l[i]):=e;
  for (i=0; i<2; i=i+1)
    for (j=0; j<2; j=j+1)
      init(rb[i][j]):=e;
  for (i=0; i<2; i=i+1)
    init(c[i]):=e;

  -- system dynamics
  for (i=0; i<4; i=i+1)
    tl[i]: process entry_exit(l[i]);

  for (i=0; i<4; i=i+1)
    for (j=0; j<2; j=j+1)
      lr[i][j]: process move(l[i],rb[(i<2?0:1)][j]);

  for (i=0; i<2; i=i+1)
    for (j=0; j<2; j=j+1)
      for (k=0; k<2; k=k+1)
        rc[i][j][k]: process move(rb[i][j],c[k]);

  for (i=0; i<2; i=i+1)
    exp[i]: process expose(c[i]);
}

module entry_exit (p)
{
  if (p=e)
    next(p):=r;
  else if (p=g)
    next(p):=e;
}

module move (lft,rgt)
{
  if (lft=r && rgt=e)
  {
    next(lft):=e;
    next(rgt):=r;
  }
  else if (lft=e && rgt = g)
  {
    next(lft):=g;
    next(rgt):=e;
  }
}

module expose (p)
{
  if (p=r)
    next(p):=g;
}

```

Fig. 2: SMV model of EUV machine.

For each of the 10 positions in the machine our model contains a state variable: an array **l** of size 4 for the locks, a 2-dimensional array **rb** of size 2×2 for the robots, and an array **c** of size 2 for the chucks. These state variables can either take value **e** (*empty*), which means that the position is empty, value **r** (*red*), which means that the position is occupied by an unexposed wafer, or **g** (*green*), which means that the position is occupied by an exposed wafer. Initially, the machine is completely empty and all state variables have value **e**.

To model the system dynamics, i.e., the movement and exposure of wafers, we introduce 22 asynchronous processes, which are executed in an interleaving fashion:

- For each of the 4 locks **i** we have process **tl[i]**, which may either put an unexposed wafer in lock **i** if it is empty, or move an exposed wafer from the lock to the track robot. In the definition of process **tl[i]** we use an auxiliary function **entry_exit** that describes the state change that results from running this process.
- For each of the 16 pairs of positions **i**, **j** such that **i** is on the left of **j** and a wafer can move directly from **i** to **j** (or back), we introduce a process that takes care of moving unexposed wafers from **i** to **j**, and exposed wafers from **j** back to **i**. In the definition of these processes we use a function **move(lft,rgt)** that describes the state change that results from moving a wafer from **lft** to **rgt** or vice versa.
- For each of the 2 chucks **i** we introduce a process **exp[i]** that models exposure of the wafer. An auxiliary function **expose** describes the state change that results from exposing the wafer at position **p**: the value of the corresponding state variable changes color from **r** (red) to **g** (green).

In the SMV model we abstract from the turning of internal robots. So a wafer can be picked up by both arms of an internal robot (possibly, the robot first has to turn). Similarly, the SMV model abstracts from chuck swaps and the measure operation. In Section 4, we present a more detailed model of the EUV machine in which we do not abstract from these aspects.

As it turns out, our SMV model has 57116 reachable states, which is close to the total number of states which equals $3^{10} = 59049$. An example of an unreachable state is one in which the machine is completely filled with exposed wafers. Transition systems of this size can very easily be handled by SMV and the computer hardware that is available today. In fact, SMV routinely handles systems with 10^{20} states and beyond, so we expect that our approach can also be applied to considerably larger designs.

3.3 Defining Deadlock and Safety in SMV

Standard textbooks on operating systems, e.g. [20], state four conditions for deadlock in systems that consist of *processes* that compete for *resources*. The first three conditions concern the model itself and are necessary, and the fourth condition concerns the states of the model and is necessary and sufficient when

the first three are met: (i) mutual exclusion: only one process may use a resource at a time, (ii) hold and wait: a process may hold allocated resources while awaiting assignment of others, (iii) no preemption: no resource can be forcibly removed from a process that is holding it, and (iv) circular wait: a closed chain of processes exists such that each process holds at least one resource needed by the next resource in the chain.

In the EUV machine, the wafers are modeled as the processes and they compete for the positions in the machine that constitute the resources. The model of the EUV machine satisfies the first three conditions for deadlock. The fourth condition, which thus is necessary and sufficient for deadlock, can be formalized with help from a *needs* function, that specifies for each wafer the set of positions it may move to. Let P denote the set of positions in the EUV machine. For $p \in P$ and $c \in \{\mathbf{r}, \mathbf{g}\}$, we define $needs(p, c) \subseteq P$ to be the set of positions (different from p) to which a wafer with color c at position p may move next. In particular, if p is a chuck, then $needs(p, \mathbf{r}) = needs(p, \mathbf{g}) = R$, where R is the set of positions of the internal robots. If s is a state and p a position then we use $needs^s(p)$ as an abbreviation for $needs(p, s(p))$. The circular wait property can now be defined as follows.

Definition 1 (Circular wait). *A state s has a circular wait in $Q \subseteq P$ iff $s(q) \neq \mathbf{e} \wedge \emptyset \neq needs^s(q) \subseteq Q \neq \emptyset$ for all $q \in Q$.*

It is not possible to directly formulate the circular wait property in terms of CTL, so some encoding is required. The basic idea is that the machine has a circular wait in a subset Q of positions iff the wafers in Q will never be able to move again. Observe that if in our model a transition $s \rightarrow s'$ moves a wafer from place p to place p' , then p is empty in s' . Thus, the property that some wafer cannot move anymore can be formalized in CTL as follows.

Definition 2 (Jam). *A position p is jammed in state s iff $s \models \mathbf{AG}(p \neq \mathbf{e})$. A state s is jammed iff some position is jammed in s .*

Proposition 1 below asserts the equivalence of the circular wait and jammed properties, thereby providing us with a way to express deadlocks in CTL. It has only been proven for our model of the EUV machine, but from the proofs it should be clear that these results can be generalized to a whole class of resource allocation problems.

Proposition 1. *A state has a circular wait in some Q iff it is jammed.*

In the remainder of this paper, we will say that a state is *deadlocked* if it has circular wait, i.e., if it is jammed. The question that we need to answer is whether and how we can prevent the system of entering a deadlocked state. In Dijkstra's paper on the *banker's algorithm* [8], the first published deadlock avoidance algorithm, a state is defined to be *safe* if "all processes can be run to completion". In our case, the wafers are the processes and "a wafer is run to completion" if it exits the machine. Thus, Dijkstra's definition can be translated to CTL as follows.

Definition 3 (Safe states). A state s is safe iff $s \models \mathbf{EF} \left(\bigwedge_{p \in P} (p = \mathbf{e}) \right)$.

Note that in general safe and not being deadlocked are different things. If a state s is not deadlocked then $s \models \bigwedge_{p \in P} \mathbf{EF}(p = \mathbf{e})$, i.e., each individual position can be emptied, but it need not be the case that all positions can be emptied simultaneously. If a state is deadlocked it is unsafe, but if it is unsafe it need not be deadlocked. However, in many cases and (according to SMV) in particular for our model of the EUV machine, the following property does hold²:

$$\mathbf{AG}(\text{safe} \iff (\mathbf{EG} \neg \text{deadlock})). \quad (1)$$

This formula suggests a simple least restrictive DAP: just keep the system in a safe state. This policy can be realized for the EUV machine. Every non-initial safe state has at least one safe successor (different from itself), otherwise it would not be not possible to return to the initial state. In addition, we verified using SMV that all successors of the initial state are again safe.

3.4 A Least Restrictive DAP

In order to actually build a controller that always keeps the system in a safe state, it would clearly be very helpful to have a simple, yet exact characterization of the set of safe states. We see two ways to obtain such a characterization.

1. When checking whether the initial state is safe, SMV computes a *binary decision diagram* (BDD, see [6]) which provides a compact representation of the set of safe states. With the available SMV releases it is not possible to get the BDD out. However, since there is an open-source distribution available solving this problem should just be a matter of programming.
2. The set of safe states can be manually characterized by the following iterative procedure:

$$\begin{aligned} S &:= \text{true} \\ \mathbf{while} \ (s_{\text{init}} \not\models \mathbf{AG}(\text{safe} \iff S)) \\ &\quad S := S \wedge (\neg C) \end{aligned}$$

where C is the characterization of the last state of the counter example that is generated by SMV.

The first approach enables a least restrictive DAP with linear time complexity, since checking whether a state is included in a BDD takes $\mathcal{O}(n)$ operations, where n is the number of booleans from which the BDD is composed (20 in case of the EUV machine). The size of the BDD, however, can in the worst case be

² In fact, in the EUV machine a state is safe if and only if it has no deadlock. It is easy to come up with variations of the machine with states that are not safe and not deadlocked, for example a design in which the internal robots only have one arm. In such cases, in order to make formula (1) hold, we need to require weak fairness for all processes in the SMV model to exclude runs in which no progress is made due to infinite stuttering of some components.

exponential in the number of booleans. A second drawback is that it can be difficult to derive individual unsafe and/or deadlock situations from a BDD, which may be required during the design phase of the system. The second approach can quickly become practically infeasible since all unsafe states are explicitly enumerated. If it is carried out manually, however, then it might be possible to abstract from irrelevant state information and to visualize the various unsafe situations in the system. Of course, this requires some effort and creativity from the analyst. The second approach has been used to characterize the safe states of the EUV machine. With five iterations, we found four unsafe situations, depicted in Figure 3, which happen to characterize all deadlocks. A right-pointing

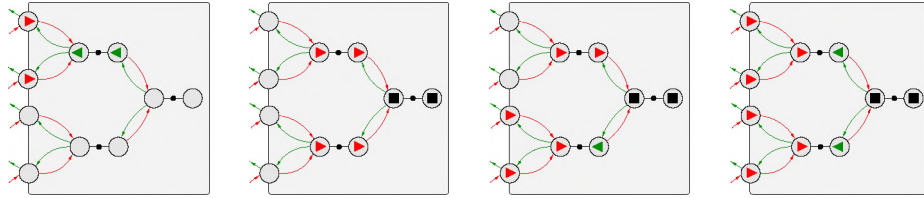


Fig. 3: The four unsafe scenarios (modulo symmetry) in the EUV machine.

arrow represents an unexposed wafer, a left-pointing arrow represents an exposed wafer, and a black square represents an unexposed or exposed wafer. The predicate S that exactly characterizes the set of safe states is the negation of the situations shown in Figure 3, and can be described in the input language of SMV with 695 characters.

Note that SMV can also be used to obtain a simple under-approximation of the set of safe states (when, e.g., the BDD is too large to use and the iterative process is too time consuming). If C is a candidate for a simple under-approximation, then this can be verified with the CTL property $\mathbf{AG}(C \Rightarrow \text{safe})$. Again, counter-examples can be used to correct C while retaining low complexity. Note, however, that it now becomes necessary to ensure that the initial state is reachable from any state in C (this is true by definition for the set of all safe states).

4 Throughput Analysis

A first objective for a controller of the EUV machine is to avoid deadlocks. In the previous section, using our SMV model, we synthesized a least restrictive control policy that achieves this. A second key objective for a controller of the machine of course is to maximize throughput. Our SMV model is not sufficiently detailed to address this issue since, for instance, relevant information about the delays in the locks and the speed of the robots has not been included. Also,

the SMV model abstracts from the delays due to turning of the internal robots, measuring of wafers, and swapping of the chucks. Therefore, in this section, we present a more refined *timed automata model* ([2, 3]), which contains sufficient information to address the throughput issue.

In order to define and analyze our model, we used the UPPAAL model checking tool. UPPAAL supports modeling of systems in terms of networks of timed automata which are extended by blocking synchronization and bounded integer variables. Similarly to SMV, the semantics of a UPPAAL model is defined by a transition system. In addition to the discrete part, the states also contain a real-valued clock valuation. For these models, the UPPAAL model checker can decide a subset of *Timed Computation Tree Logic* (TCTL, see [1]). For a detailed account of UPPAAL we refer to [13] and to <http://www.uppaal.com>.

After presenting the UPPAAL model of the EUV machine in Section 4.1, we discuss the relationship between the UPPAAL and SMV models in Section 4.2. Then, in Section 4.3, we use UPPAAL to derive a schedule for the EUV machine that optimizes throughput.

4.1 UPPAAL Model

The UPPAAL model of the EUV machine contains the same state variables as the SMV model for the positions in the machine: arrays **l**, **rb** and **c**, which may take the same values **e**, **r** and **g** to indicate that a position is respectively empty, filled with an unexposed wafer, or with an exposed wafer. In addition, the UPPAAL model has a number of Boolean state variables to ensure “physical integrity”. For instance, an internal robot can only access a lock if it is vacuum. This requirement is modeled using the Boolean **lb[id]** for lock number **id**. The model consists of 12 automata, of which 11 model physical components of the machine: the track robot, the four locks, the four robot arms (two for each of the robots), and the two chucks. These automata move wafers around with certain delays and according to the material paths as specified in Section 2. An additional automaton, the *observer*, is used for throughput optimization.

To illustrate the modeling in UPPAAL, we present the template for one arm of an internal robot, see Figure 4. This template has four parameters: a constant *id* that identifies the internal robot to which the arm belongs, two constants *l0* and *l1* that identify the locks to which the robot arm has access, and a channel *turn*. When a robot arm is at the locks, then it can get a wafer from a lock (*L02R* and *L12R*), or it can put a wafer in a lock (*R2L0* and *R2L1*). Of course, it can only perform these actions if the lock is vacuum, and if the wafer flow is as specified in Section 2. Similarly, when a robot arm is at the chucks then it can load/unload a wafer to/from the chuck that is at the *measure* location. The *cb* variables are used to ensure that only one robot arm has access to the chuck at a time and that the chuck cannot execute a transition while the robot arm is loading/unloading a wafer.

Figure 5 shows the observer process which, as we will explain in more detail in Section 4.3, is used to ensure progress in the model. This process measures the time until the first wafer exits the system (this is called an unload event) in

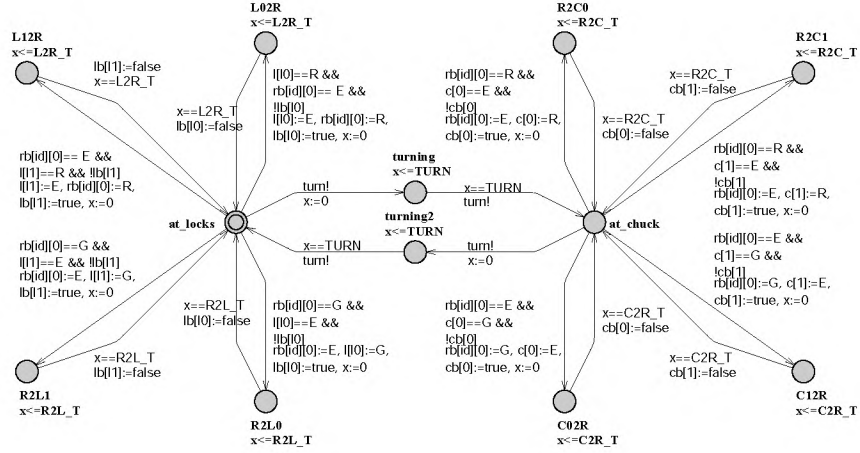


Fig. 4: Template for a robot arm.

location $L0$, and the time between two consecutive unload events in location $L1$ using its local clock x .

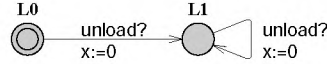


Fig. 5: Process for the observer.

4.2 Bisimulation between SMV and UPPAAL models

Clearly, there is a relationship between the SMV model and the UPPAAL model. The SMV model is an abstraction from the UPPAAL model, which has the property that every transition in the UPPAAL model can be simulated in the SMV model, and vice versa. Formally, the relationship between the two models can be expressed as a *stuttering bisimulation* relation in the sense of [5]. Stuttering bisimulations are defined in terms of *Kripke structure*, an extension of transition systems in which to each state a set of atomic propositions is associated that hold in that state.

Definition 4 (Kripke Structures). Let $\mathbf{AP}$ be a set of atomic proposition symbols. A Kripke structure is a structure $(S, s_{init}, \rightarrow, l)$, where $(S, s_{init}, \rightarrow)$ is a transition system and function $l : S \rightarrow 2^{\mathbf{AP}}$ associates to each state a set of atomic proposition symbols.

In this paper, we let $\mathbf{AP}$ be the set of equations of the form $p = v$, where p is a position in the EUV machine and $v \in \{\mathbf{e}, \mathbf{r}, \mathbf{g}\}$. For the transition systems induced by the SMV and UPPAAL models, the labeling is obvious: we label a state s with $p = v$ iff this equation holds in s . For the SMV model the labeling

function is injective: different states have different labels. For the UPPAAL model this is clearly not the case.

A stuttering bisimulation relates states from two Kripke structures. Initial states are related, and related states are labeled with the same proposition symbols. If two states are related and from one state a transition is possible, then it should be possible to simulate this transition from the related state, after first doing zero or more *stuttering transitions*, i.e., transitions that do not change the labeling.

Definition 5 (Stuttering Bisimulation). A stuttering bisimulation *between Kripke structures* $(S, s_{init}, \rightarrow, l)$ and $(S', s'_{init}, \rightarrow', l)$ is a relation $R \subseteq S \times S'$ s.t.

1. $(s_{init}, s'_{init}) \in R$,
2. If $(r, s) \in R$ then $l(r) = l(s)$,
3. if $(r, s) \in R$ and $r \rightarrow r'$ then there exist, for some $n \geq 0$, $s_0, s_1, \dots, s_n$ such that $s_0 = s$ and, for all $i < n$, $s_i \rightarrow' s_{i+1}$, $(r, s_i) \in R$ and $(r', s_n) \in R$.
4. if $(r, s) \in R$ and $s \rightarrow s'$ then there exist, for some $n \geq 0$, $r_0, r_1, \dots, r_n$ such that $r_0 = r$ and, for all $i < n$, $r_i \rightarrow r_{i+1}$, $(r_i, s) \in R$ and $(r_n, s') \in R$.

Proposition 2. Consider the projection function π from states of the Kripke structure induced by the UPPAAL model to states of the Kripke structure induced by the SMV model. Function π only preserves the values of the arrays **l**, **rb** and **c**. Let R be the relation consisting of pairs $(s, \pi(s))$, for s a reachable state from the UPPAAL model. Then R is a stuttering bisimulation between the UPPAAL and SMV Kripke structures.

The significance of the above result stems from the fact that validity of CTL formulas without *nexttime* operator (i.e. all the formulas used in this paper) is preserved by stuttering bisimulation equivalence (see [5]). Thus, all the results on deadlock avoidance established using SMV in Section 3 carry over to the UPPAAL model. It is not possible to obtain these results directly using the UPPAAL tool since (a) UPPAAL does not support full CTL, and (b) the state space of the UPPAAL model is so big that it cannot be fully explored.

4.3 Finding an Optimal Schedule

As mentioned above, the *observer* process of Figure 5 observes unload events. It starts in location *L0* and upon the first unload event it resets its local clock x and enters location *L1*. In location *L1* the clock is reset whenever an unload event takes place. The observer is used to find an infinite schedule that takes at most H time units until the first unload event, and that has at most S time units between two unload events. Such a schedule is specified by the following TCTL property that can be checked by UPPAAL.

$$\mathbf{EG}((\text{observer.L0} \Rightarrow \text{observer.x} \leq H) \wedge (\text{observer.L1} \Rightarrow \text{observer.x} \leq S)) \quad (2)$$

If this property is satisfied, then UPPAAL can return an example execution that consists of a path followed by a cycle. Such an execution thus gives an infinite

control schedule for the wafer scanner with a *stationary* throughput of at least one wafer per S time units. Unfortunately, the size of the reachable state space prevents UPPAAL from finding such an execution directly. We therefore added heuristics to the model to prune the state space:

1. The DAP derived in the previous section has been used to avoid unsafe material configurations of the machine.
2. Some transitions are useless (or suboptimal) in certain states, e.g., an internal robot can always turn, but this is useless if it does not hold wafers. The state space has been reduced by adding guards that prevent such useless behavior.
3. The optimal behavior of the locks in the initial phase (the filling of the machine) differs from their optimal behavior in the stationary phase. Therefore a heuristic has been added to enforce this difference: a lock can pressurize when it contains either an exposed wafer, or it is empty and the machine is not yet filled with enough wafers to be in the stationary state.
4. Some transitions have been made *urgent* (greedy): they must be taken as soon as they are enabled. For instance, if the DAP allows loading a wafer to a lock, then this must be done immediately.

Note that using urgent transitions without the DAP may be an unwise idea, since this can result in many deadlocks with the effect that an execution satisfying Property 2 does not exist anymore in the model. Also note that at least the last three heuristics may remove good schedules.

A lower bound on the time until the first unload event, min_h , can easily be derived from the model. It is also easy to see that the minimal separation time between exposed wafers that appear at the chuck that is in the measure position (and can therefore be picked up by an internal robot) equals $min_s = EXPOSE + SWAP$, where the former is the time needed for the expose operation and the latter is the time needed for the chuck swap. Therefore, the theoretical maximal stationary throughput of the machine is at most one wafer per min_s time units. For the UPPAAL model with heuristics it is possible to find an execution that satisfies Property 2 for a value of H that is 5% larger than min_h and for $S = min_s$. Figure 6 shows this schedule that optimizes the stationary throughput of the EUV machine.

It took only little effort to change the UPPAAL model in order to analyze two alternative machine designs w.r.t. throughput. In the first design, the incoming wafers have been restricted to the upper two locks and the outgoing wafers to the lower two locks (to prevent deadlock a priori; see Section 2). We can easily find an optimal schedule with $S = 1.61 \cdot min_s$ that shows that not the expose operation but the locks have become critical. This confirms our suspicion that has been stated in Section 2. The second alternative design consists of only two locks and one internal robot. We can easily find a schedule with $S = 1.82 \cdot min_s$, but we cannot guarantee that this is an optimal schedule.

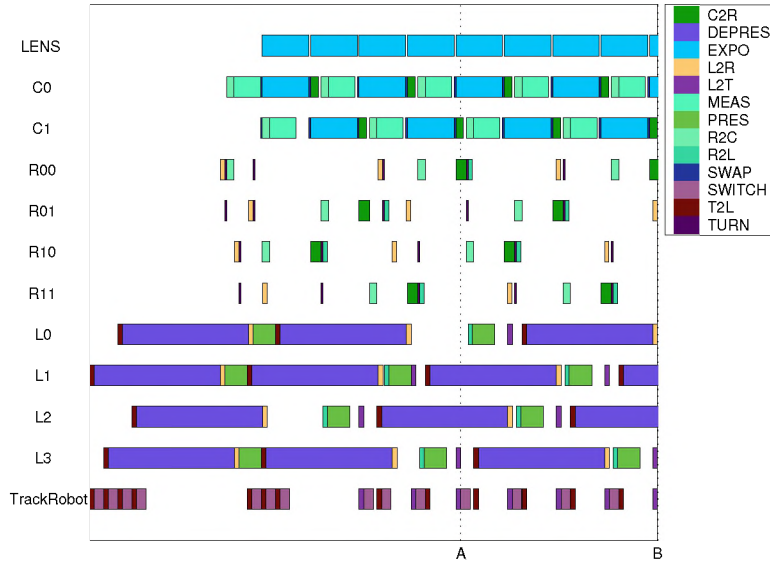


Fig. 6: A schedule that optimizes the stationary throughput of the EUV machine. The cyclic part of the schedule consists of the interval between points A and B. Note that the operation of the lens is only interrupted by the chuck swap (which is necessary).

5 Conclusions

The SMV model checker has successfully been used to characterize the set of safe states of the EUV machine. This characterization consists of a very short boolean expression over the places in the machine and is useful for the design of an actual controller since deadlock can easily be avoided by examining the possible successor states of the current state. Since the characterization is exact, the controller implements a least restrictive (optimal) deadlock avoidance policy. Furthermore, we used the UPPAAL model checker to compute infinite schedules for the EUV machine that optimize stationary throughput. It took little effort to change the UPPAAL model in order to analyze two alternative machine designs. In theory, our approach can be applied to a broad class of resource allocation systems. As always when using model checking, the state space explosion is the main problem for scalability. Altogether, in our view, the present work nicely illustrates the usefulness of model checking techniques to support the design process of applications that involve resource allocation and scheduling. Building models that are just abstract enough for addressing a specific question, often provides a good way to deal with the state space explosion problem.

Acknowledgements. The authors thank Biniam Gebremichael for his useful suggestions concerning the SMV model, and the anonymous reviewers for their helpful comments on a preliminary version of the present paper.

References

1. R. Alur, C. Courcoubetis, and D. L. Dill. Model checking in dense real time. *Information and Computation*, 104:2–34, 1993.
2. R. Alur and D. L. Dill. Automata for modeling real-time systems. In *Proceedings 17th ICALP*, pages 322–335, 1990.
3. R. Alur and D. L. Dill. A theory of timed automata. *TCS*, 126:183–235, 1994.
4. N. C. W. M. Braspenning. Scheduling and behavior verification of machines based on task-resource models. Master’s thesis, Department of Mechanical Engineering, Eindhoven University of Technology, The Netherlands, October 2003. Confidential.
5. M.C. Browne, E.M. Clarke, and O. Grumberg. Characterizing finite Kripke structures in propositional temporal logic. *TCS*, 59(1,2):115–131, 1988.
6. R. E. Bryant. Graph-based algorithms for boolean function manipulation. *IEEE Transaction on Computers*, C-35(8):677–691, August 1986.
7. E. M. Clarke, O. Grumberg, and D. A. Peled. *Model Checking*. MIT Press, 2000.
8. E. W. Dijkstra. Cooperating sequential processes. Technical report, Eindhoven University of Technology, The Netherlands, 1965.
9. A. Fehnker. Scheduling a steel plant with timed automata. In *Proceedings RTCSA’99*. IEEE Computer Society Press, 1999.
10. B. Gebremichael and F. W. Vaandrager. Control synthesis for a smart card personalization system using symbolic model checking. In *Proceedings FORMATS’03*, LNCS 2791, pages 189–203. Springer-Verlag, 2004.
11. V. Hartonas-Garmhausen, E. M. Clarke, and S. Campos. Deadlock prevention in flexible manufacturing systems using symbolic model checking. In *IEEE Conference on Robotics and Automation*, volume 1, pages 527–532, 1996.
12. M. Hendriks, N.J.M. van den Nieuwelaar, and F.W. Vaandrager. Model checker aided design of a controller for a wafer scanner. Report NIII-R0430, Institute for Computing and Information Sciences, University of Nijmegen, June 2004.
13. K. G. Larsen, P. Pettersson, and W. Yi. UPPAAL in a nutshell. *International Journal on Software Tools for Technology Transfer*, 1(1/2):134–152, 1997.
14. M. Lawley and S. A. Reveliotis. Deadlock avoidance for sequential resource allocation systems: Hard and easy cases. *International Journal of Flexible Manufacturing Systems*, 13(4):385–404, 2001.
15. M. Lawley, S. A. Reveliotis, and P. Ferreira. Design guidelines for deadlock handling strategies in flexible manufacturing systems. *International Journal of Flexible Manufacturing Systems*, 9(1):5–30, January 1997.
16. K. L. McMillan. *Symbolic Model Checking*. PhD thesis, Carnegie Mellon University, Pittsburgh, May 1992.
17. T. Murata. Petri nets: Properties, analysis, and applications. *Proceedings of the IEEE*, 77(4):541–580, 1989.
18. J. Park and S. A. Reveliotis. Deadlock avoidance in sequential resource allocation systems with multiple resource acquisitions and flexible routings. *IEEE Transactions on Automatic Control*, 46(10):1572–1583, 2001.
19. S. A. Reveliotis, M. Lawley, and P. Ferreira. Polynomial-complexity deadlock avoidance policies for sequential resource allocation systems. *IEEE Transactions on Automatic Control*, 42(10):1344–1357, 1997.
20. W. Stallings. *Operating Systems*. Prentice-Hall, 1998.
21. Y. Wang and Z. Wu. Deadlock avoidance control synthesis in manufacturing systems using model checking. In *IEEE American Control Conference*, volume 2, pages 1702–1704, 2003.
22. S. Yovine. KRONOS: a verification tool for real-time systems. *International Journal on Software Tools for Technology Transfer*, 1(1/2):123–133, 1997.